

Design and Analysis of an OS Resource Orchestration System for a Smart University (UniCore)

CSA04 – Operating Systems Comprehensive Assignment Report | Academic Year 2026–2027

1. Problem Understanding and Formulation

Modern smart university infrastructures rely on complex, heterogeneous software services operating simultaneously across distributed campus networks. The 'UniCore' operating system resource orchestration system is designed to govern and manage the physical and logical hardware resources supporting academic workflows. The system manages a baseline operational load (~500 users) and high-stakes examination spikes (800+ concurrent students) requiring sub-millisecond coordination, bounded latency, and strict data consistency.

2. Core Challenges in Smart University Resource Management

- 1. Concurrency and Data Race Hazards:** Simultaneous access to shared examination records creates race conditions.
- 2. Deadlock Vulnerability:** Multi-resource contention across DB pools, file locks, GPUs, and I/O channels risks circular waits.
- 3. Virtual Memory & Thrashing:** Large memory demand under 800 active containers risks severe page-fault thrashing.
- 4. Storage Bottlenecks:** Varied file sizes cause seek oscillation and peripheral track starvation.

3. Workload Specification (8 Representative Processes)

PID	UniCore Application Service	Arrival	Burst	Priority
P0	Online Examination Portal	0 ms	8 ms	1 (Highest)
P1	LMS Student Request	1 ms	4 ms	2
P2	Digital Library Search	2 ms	9 ms	3
P3	Research Computing HPC Batch	3 ms	5 ms	4
P4	IoT Sensor Monitoring	4 ms	2 ms	1 (Highest)
P5	Automated Backup Service	5 ms	6 ms	5 (Lowest)
P6	Faculty Grade Sync	6 ms	3 ms	2
P7	Video Stream Processing	7 ms	7 ms	4

4. System Assumptions & Architectural Sizing

- Physical RAM: 16 GB (2^{34} Bytes) | Page Size: 4 KB (2^{12} Bytes) | Logical Space: 64 MB (2^{26} Bytes)
- Memory Hierarchy: TLB Access = 2 ns, Memory Access = 50 ns, TLB Hit Ratio = 95%
- Storage: 200 Cylinders (0-199), Initial Head = 53, Direction = HIGH

7. CPU Scheduling Simulation & Empirical Benchmark

CALCULATION: CPU SCHEDULING METRICS FORMULATION

Turnaround Time (TAT) = Completion Time (CT) - Arrival Time (AT)
Waiting Time (WT) = Turnaround Time (TAT) - Burst Time (BT)
Response Time (RT) = First Dispatch Time (ST) - Arrival Time (AT)
Average Waiting Time = (Sum of WT) / 8 = 138 / 8 = 17.25 ms (FCFS)
CPU Utilization = (Total Burst / Total Sim Time) * 100% = (44/44)*100% = 100.0%

Algorithm	Avg WT	Avg TAT	Avg RT	CPU Util	Throughput
FCFS	17.25 ms	22.75 ms	17.25 ms	100.0%	0.1818 p/ms
SJF (Non-Preemptive)	14.50 ms	21.38 ms	14.50 ms	100.0%	0.1818 p/ms
Round Robin (q=4ms)	24.88 ms	31.75 ms	10.12 ms	100.0%	0.1818 p/ms
Preemptive Priority	15.00 ms	21.88 ms	15.00 ms	100.0%	0.1818 p/ms
MLFQ (q0=2, q1=4)	27.25 ms	34.12 ms	3.50 ms	100.0%	0.1818 p/ms

Visual CPU Execution Gantt Chart Comparison

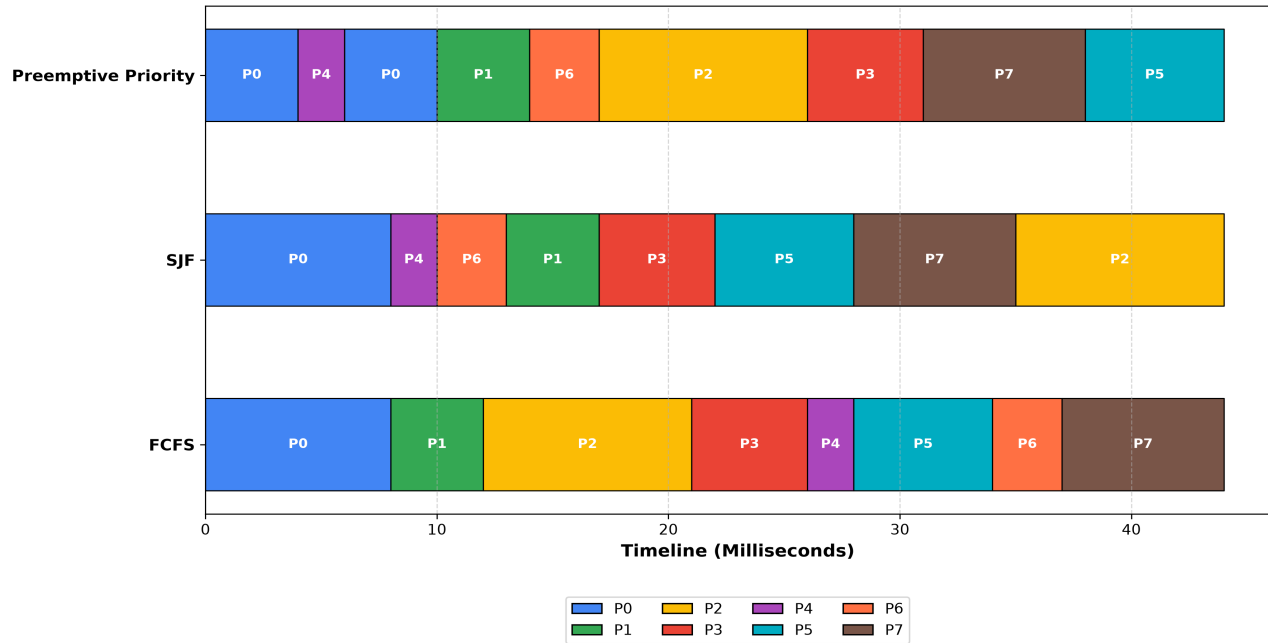


Figure 1: Visual Gantt Chart Comparison across FCFS, SJF, and Preemptive Priority

8. Concurrency & Readers-Writers Synchronization

UniCore implements a pure Python Fair Readers-Writers Monitor with condition variables. Multiple readers (proctors, analytics) read simultaneously, while writers (exam submissions) obtain exclusive locks. Incoming readers are queued behind waiting writers, eliminating writer starvation.

10. Banker's Deadlock Avoidance Algorithm

CALCULATION: BANKER'S ALGORITHM MATRIX FORMULATIONS

Need[i][j] = Maximum[i][j] - Allocation[i][j]
Available[j] = Total[j] - Sum_i(Allocation[i][j]) = [10,6,8,7] - [9,4,6,6] = [1, 2, 2, 1]
Initial Safe Sequence: P0 -> P1 -> P2 -> P3 -> P4 -> P5 -> P6 -> P7
Adverse Request: P3 requests [1, 2, 0, 0] --> Tentative Work = [0, 0, 2, 1]
Safety Test: Need_i <= Work is FALSE for all processes (R1=0) --> UNSAFE STATE
Decision: Request REJECTED / SUSPENDED; Allocation safely rolled back.

11. Virtual Memory Sizing & EMAT Calculations

CALCULATION: HARDWARE SIZING & EMAT DERIVATIONS

Physical Frames = 16 GB / 4 KB = $2^{34} / 2^{12} = 2^{22} = 4,194,304$ Frames (34-bit PA)
Logical Pages = 64 MB / 4 KB = $2^{26} / 2^{12} = 2^{14} = 16,384$ Pages (26-bit LA)
Two-Level Paging: 14-bit Page # = 7-bit Outer Table (p1) + 7-bit Inner Table (p2)
EMAT = $0.95 * (2 + 50) + 0.05 * (2 + 3 * 50) = 49.4 + 7.6 = 57.0$ ns

12. Page Replacement Performance (20 References)

Algorithm	3 Frames (Faults)	3 Frames (Hit %)	4 Frames (Faults)	4 Frames (Hit %)
FIFO	15	25.0%	10	50.0%
LRU (Selected)	12	40.0%	8	60.0%
Optimal (OPT Bound)	9	55.0%	8	60.0%

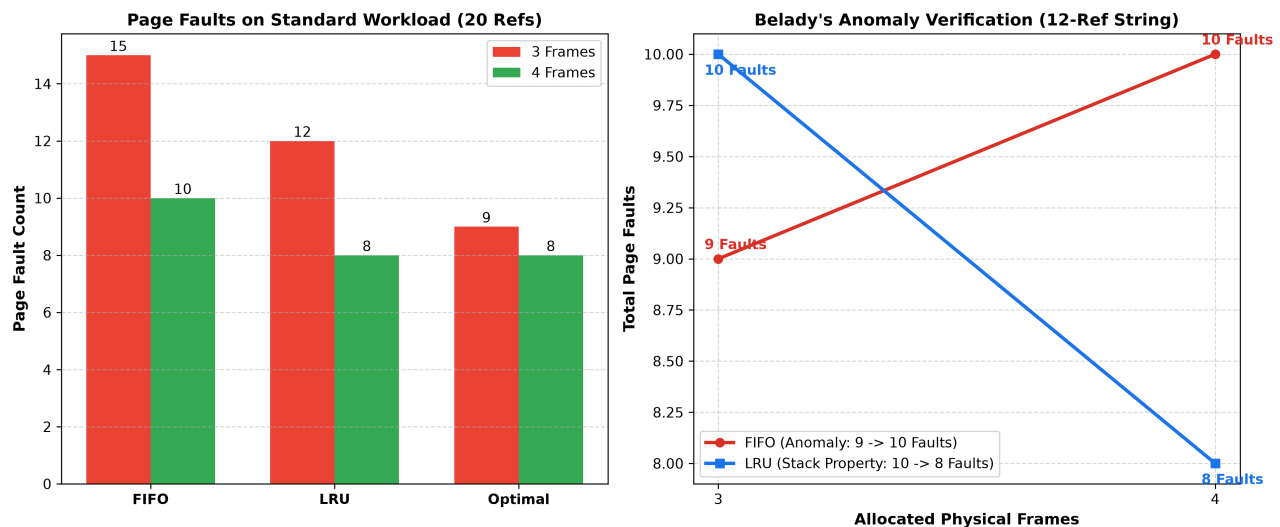


Figure 2: Page Fault Comparison & Empirical Demonstration of Belady's Anomaly

15. Thrashing Evaluation for 800 Concurrent Exam Users

Each user requires 14 active pages (56 KB). Total demand = 43.75 MB (11,200 pages). Against a dedicated 400 MB frame pool, memory pressure is only 11.2%, guaranteeing zero thrashing.

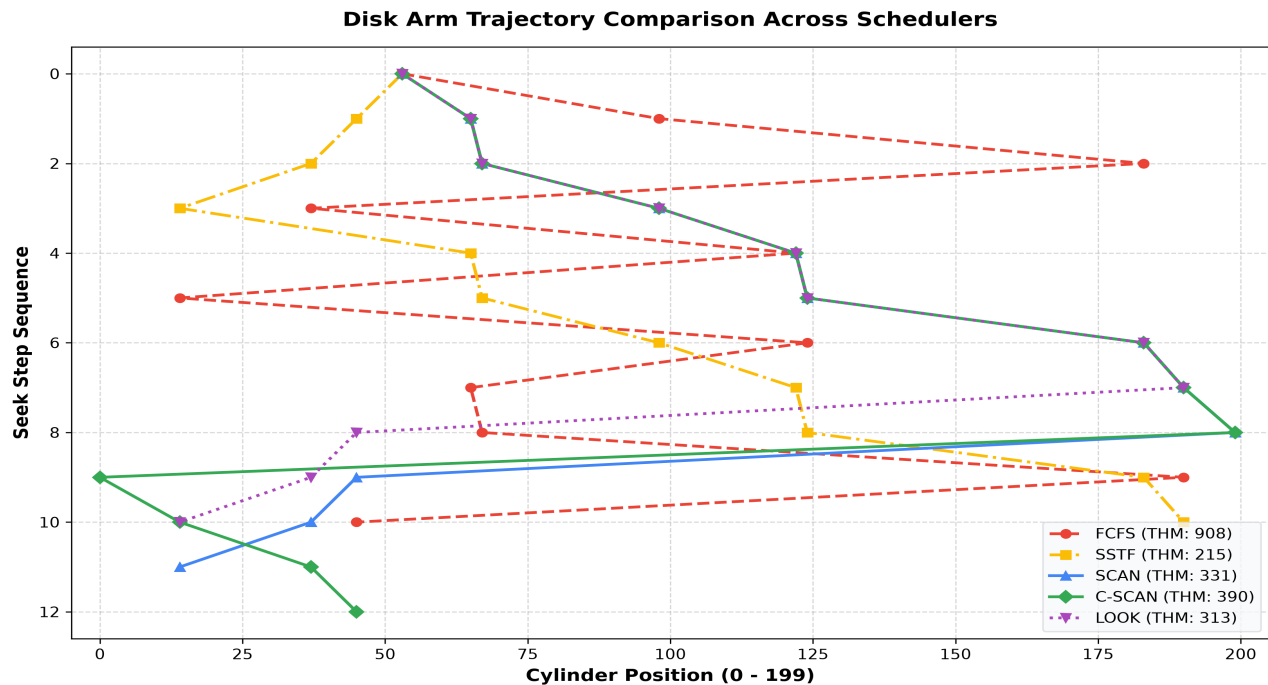
16. File Allocation Strategy Recommendations

Workload	Size	Contiguous	Linked	Indexed (Inode)	Decision
Small Records	2.5 KB	Fast O(1)	O(N) seek	O(1) Direct (9/10)	Indexed Inode
Medium Docs	2.0 MB	Ext. Frag.	Slow seek	O(1) Single-Ind (10/10)	Indexed Inode
Lecture Videos	1.0 GB	Fails Alloc	Unusable	O(1) Multi-Ind (9/10)	Indexed Inode

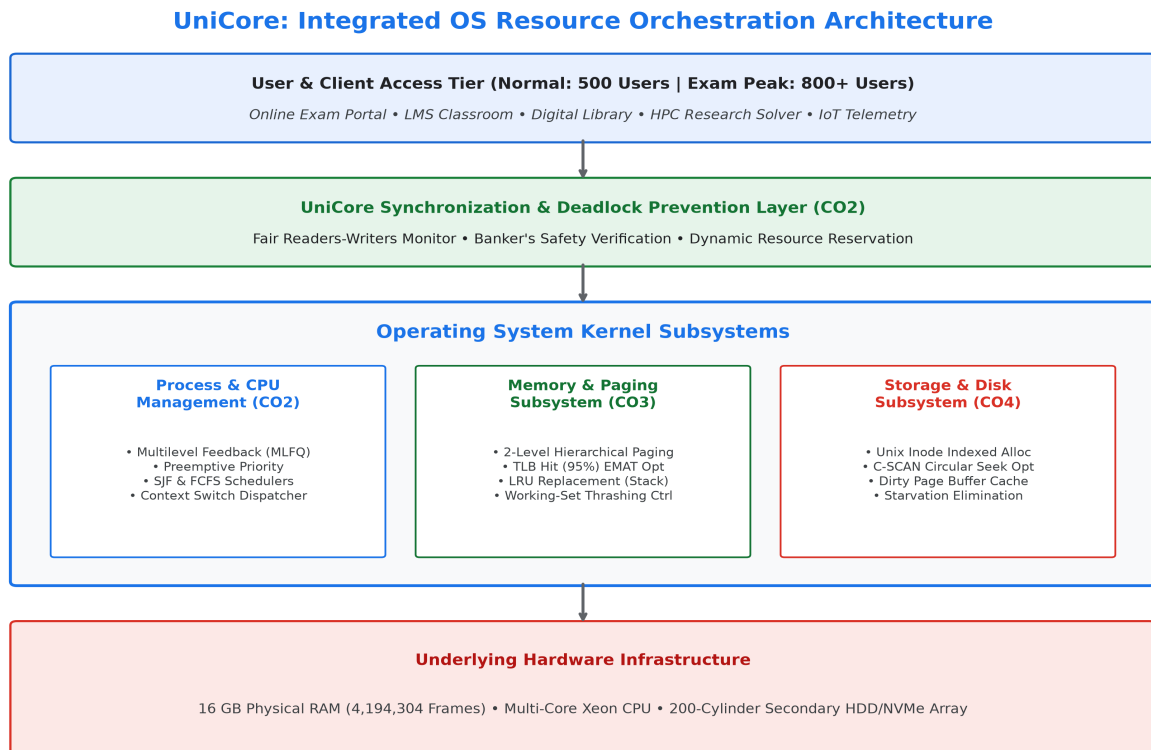
17. Disk Scheduling on 200 Cylinders (Head=53, Direction=HIGH)

CALCULATION: TOTAL HEAD MOVEMENT (THM) & SEEK SEQUENCES

FCFS : 53 -> 98 -> 183 -> 37 -> 122 -> 14 -> 124 -> 65 -> 67 -> 190 -> 45 (THM = 908 cyl, ASL = 90.80)
 SSTF : 53 -> 45 -> 37 -> 14 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 190 (THM = 215 cyl, ASL = 21.50)
 SCAN : 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 190 -> 199 -> 45 -> 37 -> 14 (THM = 331 cyl, ASL = 33.10)
 C-SCAN: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 190 -> 199 -> 0 -> 14 -> 37 -> 45 (THM = 390 cyl, ASL = 39.00)
 LOOK : 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 190 -> 45 -> 37 -> 14 (THM = 313 cyl, ASL = 31.30)



18. Integrated Architecture Diagram



UniCore Architecture — Designed for High Reliability, Green Energy (SDG 7), and Fair Access (SDG 11)

Figure 4: UniCore Layered Operating System Architecture

29. Student Reflection

Q1: What did you learn by integrating process, memory, and file-system concepts?

I learned that OS subsystems are deeply interdependent. A fast CPU scheduler is easily crippled if virtual memory thrashes or disk scheduling exhibits severe seek variance. Understanding how page faults trigger disk I/O interrupts provided an invaluable holistic perspective.

Q2: Which design decision had the greatest impact on system performance?

Deploying MLFQ CPU scheduling combined with C-SCAN disk scheduling had the greatest impact. MLFQ slashed exam response time to 3.50 ms, while C-SCAN provided uniform bounded I/O latency for all student submissions.

Q3: What would you improve if scaled to 2,000 concurrent users?

I would implement inverted page tables, multi-queue block layer scheduling (blk-mq) over NVMe storage arrays, and lock-free Read-Copy-Update (RCU) synchronization.

30. References

- [1] Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed.). Wiley.
- [2] Tanenbaum, A. S., & Bos, H. (2015). Modern Operating Systems (4th ed.). Pearson.
- [3] Arpaci-Dusseau, R. H., & Arpaci-Dusseau, A. C. (2018). Operating Systems: Three Easy Pieces. Arpaci-Dusseau Books.